Herald

Protocol Research: CloudEvents

Field	Value
Revision	1
Created	2026-05-22
Last modified	2026-05-22
Status	research-only (partially in use — formalize bindings)
Status summary	CloudEvents v1.0.2 is the CNCF standard for event data envelopes. Herald ALREADY uses CloudEvents envelopes internally (`commons.CloudEventEnvelope` exists). Wave 4c should formally implement the HTTP binding (binary + structured modes) on every `/v1/events` ingest route + cement the JSON event-format contract. Other bindings (Kafka, AMQP) are opt-in via separate per-broker protocol adoption (see KAFKA.md, AMQP.md).
Issues	open-questions: which HTTP content mode is default; how `ce-traceparent` extension fits with W3C TraceContext; required vs optional ce-attributes.
Continuation	Wave 4c — open HRD-251..HRD-260; formalize HTTP binding; add ce-extension catalog to spec V3.

Constitutional anchors

- **§107 anti-bluff** — CloudEvents tests assert byte-level envelope conformance (binary mode: ce-* headers; structured mode: JSON body shape).
- **§11.4.74 catalogue-check** — performed; `cloudevents/sdk-go` is the canonical Go library and Herald already references CloudEvents in `commons/types.go`. No `vasic-digital` or `HelixDevelopment` CloudEvents module exists. Verdict: vendor `cloudevents/sdk-go` as submodule.
- **§11.4.61 / branding** — tracked doc; regenerate siblings.

Table of contents

- §1. Protocol overview
- §2. Specification deep-dive
- §3. Herald-specific applicability analysis
- §4. Step-by-step implementation guide for Herald
- §5. §107 anti-bluff testing strategy

- §6. Open questions for operator
- §7. References
- §8. Catalogue-check verdict

§1. Protocol overview

CloudEvents is a vendor-neutral specification for describing event data in a common, structured way. Current version: **v1.0.2** (released 2022, stable; backward-compat policy preserved). CNCF Graduated project. License: Apache 2.0. Spec repo: https://github.com/cloudevents/spec.

Scope. CloudEvents defines the EVENT FORMAT — the attributes (`id`, `source`, `specversion`, `type`, plus optional `time`, `datacontenttype`, `subject`, `dataschema`, `data`), the JSON serialization, and PROTOCOL BINDINGS for HTTP, Kafka, AMQP, MQTT, NATS, WebSockets, and others. It does NOT define a transport — it defines how events are CARRIED over existing transports.

Key insight for Herald. CloudEvents is NOT a transport protocol — it's an envelope spec. Herald uses CloudEvents envelopes on top of HTTP (REST + MCP-over-HTTP + A2A); will use it on top of Kafka when KAFKA.md is adopted; will use it on top of AMQP when AMQP.md is adopted. ONE envelope; N transports.

§2. Specification deep-dive

§2.1 Required attributes

Attribute	Type	Description
`id`	string	event identifier (idempotency key); MUST be unique within `source`
`source`	URI-reference	the producer; e.g. `https://pherald.example.com/tenants/{tenant_id}`
`specversion`	string	always `1.0` for v1.0.x
`type`	string	event type; e.g. `com.example.incident.created.v1`

§2.2 Optional attributes

Attribute	Type	Description
`datacontenttype`	string	content type of `data` (`application/json` default)
`dataschema`	URI	URI to JSON Schema describing `data`
`subject`	string	subject of the event in context of producer
`time`	RFC 3339 timestamp	when event occurred at source
`data`	any	the event payload

§2.3 Extensions

Extensions live alongside core attributes. Examples:

- `traceparent` / `tracestate` — W3C TraceContext (de-facto standard CloudEvents extension).
- `partitionkey` — for Kafka partitioning.
- `dataref` — opaque URI when data is too large to embed (data lakes pattern).
- `time-sequenced` (proposed) — strict ordering.

§2.4 HTTP binding — two modes

Binary mode (default for HTTP — most efficient):

- Each CloudEvent attribute → HTTP header prefixed `ce-` (e.g. `ce-id`, `ce-source`, `ce-type`, `ce-specversion`).
- `data` → HTTP body, as-is.
- `Content-Type` HTTP header = `datacontenttype` (e.g. `application/json`).
- Extensions go to `ce-<extname>` headers, percent-encoded if non-ASCII.

Example POST request (binary mode):

```
POST /v1/events HTTP/1.1
Content-Type: application/json
ce-specversion: 1.0
ce-id: 5f8e0c1a-...
ce-source: https://pherald.example.com/tenants/abc
ce-type: com.example.incident.created.v1
ce-time: 2026-05-22T14:32:00Z
ce-traceparent: 00-...
{
  "incident_id": "4321",
  "severity": "P1",
  ...
}
```

Structured mode (CloudEvent serialized into the body as a complete JSON document):

- HTTP body = the full CloudEvent JSON (including attributes AND data).
- `Content-Type` = `application/cloudevents+json`.
- No `ce-*` headers.

Example POST (structured mode):

```
POST /v1/events HTTP/1.1
Content-Type: application/cloudevents+json
{
  "specversion": "1.0",
  "id": "5f8e0c1a-...",
  "source": "https://pherald.example.com/tenants/abc",
  "type": "com.example.incident.created.v1",
  "time": "2026-05-22T14:32:00Z",
  "data": { "incident_id": "4321", "severity": "P1" }
}
```

§2.5 JSON event format

Defined in the JSON Event Format spec. Key rules:

- Attribute names lowercase.
- `data` MAY be any JSON value (object, array, string, number, boolean, null).
- For binary data (non-JSON), use `data_base64` instead of `data`.

§2.6 Kafka binding

Each Kafka message:

- Attributes → Kafka headers prefixed `ce_` (note: underscore for Kafka, not dash).
- `data` → Kafka value.
- `partitionkey` extension → Kafka partition key.

§2.7 AMQP 1.0 binding

Attributes → AMQP application properties prefixed `cloudEvents:`. `data` → AMQP message body.

§2.8 NATS / MQTT bindings

Similar pattern: attributes prefixed in the protocol-specific metadata channel; `data` in the message body.

§3. Herald-specific applicability analysis

§3.1 Current state

Herald already declares `commons.CloudEventEnvelope` (per `commons/types.go`) and uses CloudEvents semantics in its `events_processed` table (event_id → `ce-id`, source URI, type, time). The CloudEvents library `cloudevents/sdk-go` is NOT yet a Herald dependency — Herald uses ad-hoc field naming.

§3.2 What Wave 4c formalizes

1. **HTTP binding implementation.** Every `/v1/events` ingest route accepts BOTH binary mode and structured mode; Herald exposes a `Content-Type: application/cloudevents+json` accept-list.
2. **Outbound CloudEvents.** When Herald POSTs to user webhooks (Wave 4d), Herald emits CloudEvents binary mode by default.
3. **Type registry.** Spec V3 §4z catalogs every Herald event type:
    - `com.helix.herald.event.ingested.v1`
    - `com.helix.herald.notification.sent.v1`
    - `com.helix.herald.incident.created.v1`
    - `com.helix.herald.compliance.attested.v1`
    - ... etc.

§3.3 ce-traceparent extension

Herald already wants W3C TraceContext. The `ce-traceparent` extension is the de-facto standard for CloudEvents trace propagation. Adopt directly; no Herald-specific extension needed.

§3.4 Multi-tenant

`tenant_id` is part of the `source` URI: `https://pherald.example.com/tenants/{tenant_id}`. Document this convention in spec V3 §4z.

§3.5 Idempotency

`ce-id` IS the idempotency key. Herald's Redis SETNX path uses `ce-id` directly. No additional extension needed.

§4. Step-by-step implementation guide for Herald

§4.1 Vendor the SDK

```
git submodule add git@github.com:cloudevents/sdk-go.git
        submodules/go-cloudevents-sdk
git -C submodules/go-cloudevents-sdk checkout v2.16.0
```

Alternative: `go get github.com/cloudevents/sdk-go/v2@v2.16.0`. Operator decides per §9.1.

§4.2 Refactor `commons.CloudEventEnvelope`

Replace the ad-hoc struct with `cloudevents/sdk-go/v2/event.Event`. Migration in `commons/types.go` + sweep all consumers.

§4.3 HTTP middleware

New `commons/http/cloudevents.go`:

```
// CloudEventsMiddleware parses incoming HTTP requests into
        ce.Event
// supporting BOTH binary and structured modes per CloudEvents
        1.0.2 §3.
func CloudEventsMiddleware(next gin.HandlerFunc) gin.HandlerFunc
        { ... }
```

Wires into every `/v1/events` route + the future `/v1/webhooks/inbound` (Wave 4d).

§4.4 Outbound

New `commons/http/cloudevents_send.go`:

```
// SendCloudEvent posts an event in binary mode to the given URL
        with HMAC sig.
func SendCloudEvent(ctx, url string, ev ce.Event, signingKey
        []byte) error { ... }
```

§4.5 Event type catalog (spec V3 §4z)

Per Herald-flavor section:

- pherald — `com.helix.pherald.event.ingested.v1`, `com.helix.pherald.notification.sent.v1`.
- cherald — `com.helix.cherald.compliance.attested.v1`, `com.helix.cherald.evidence.collected.v1`.
- sherald — `com.helix.sherald.safety.event.v1`, `com.helix.sherald.oom.killed.v1`.
- bherald — `com.helix.bherald.budget.overrun.v1`.
- rherald — `com.helix.rherald.risk.flagged.v1`.
- iherald — `com.helix.iherald.incident.created.v1`, `com.helix.iherald.incident.resolved.v1`.
- scherald — `com.helix.scherald.job.scheduled.v1`, `com.helix.scherald.job.completed.v1`.

§4.6 New e2e_bluff_hunt invariants

- **E64** — binary mode round-trip: POST CloudEvent with ce-* headers → Herald stores it → GET via REST returns equivalent CloudEvent.
- **E65** — structured mode round-trip: POST `application/cloudevents+json` body → same outcome.
- **E66** — extension propagation: `ce-traceparent` header travels through Herald and is emitted on the outbound webhook (Wave 4d coupling).

§4.7 HRD scaffolding

- **HRD-251** — vendor cloudevents/sdk-go.
- **HRD-252** — replace `commons.CloudEventEnvelope` with sdk-go's Event type.
- **HRD-253** — implement HTTP binding middleware.
- **HRD-254** — emit outbound CloudEvents on webhooks.
- **HRD-255** — event type catalog (spec V3 §4z).
- **HRD-256** — ce-traceparent extension wiring.
- **HRD-257** — e2e_bluff_hunt E64–E66.
- **HRD-258** — mutation gates.
- **HRD-259** — operator credentials guide update (no creds needed, just doc).
- **HRD-260** — close Wave 4c.

§5. §107 anti-bluff testing strategy

The bar: any standard CloudEvents consumer (Knative Eventing, Argo Events, etc.) can POST a CloudEvent to Herald (binary or structured mode) and Herald processes it correctly. Any standard CloudEvents producer can RECEIVE a Herald-emitted CloudEvent and parse it correctly.

§5.1 Happy-path tests

- **Test 1: binary mode ingest.** `curl -X POST -H 'ce-specversion: 1.0' -H 'ce-id: ...' ... /v1/events` → 202 + event landed in DB with correct `ce-id` as event_id.
- **Test 2: structured mode ingest.** `curl -X POST -H 'Content-Type: application/cloudevents+json' --data @event.json /v1/events` → same outcome.
- **Test 3: outbound.** Configure a test webhook; trigger an event; assert the outbound POST has correct `ce-*` headers.
- **Test 4: standard SDK consumer.** A separate Go program imports `cloudevents/sdk-go/v2` and POSTs to Herald using the SDK's HTTP client. Asserts no validation errors.
- **Test 5: standard SDK producer.** A Go program imports the SDK and receives Herald's outbound CloudEvent. Asserts the Event parses without error.

§5.2 Edge cases

- **Test 6: extension passthrough.** `ce-traceparent` extension on inbound flows through to outbound.
- **Test 7: data_base64.** Binary data (non-JSON) via `data_base64` field; round-trip without corruption.
- **Test 8: missing required attr.** `ce-id` missing → 400.
- **Test 9: malformed specversion.** `ce-specversion: 2.0` → 400.

§5.3 Mutation gates

- Mutation: strip `ce-id` from middleware → Test 1 FAILS.
- Mutation: skip `application/cloudevents+json` content-type detection → Test 2 falls back to binary mode and FAILS (no ce-* headers).

§5.4 Wire-level inspection

- tcpdump on :7104 during Test 1 → assert `ce-*` headers present.
- Compare wire format against `golden/cloudevents/binary_mode.txt` byte-by-byte.

§6. Open questions for operator

1. **Default content mode for inbound?** Binary (efficient) or structured (easier to debug)? Recommend: accept BOTH; default to BINARY for outbound.
2. **Event-type naming convention?** `com.helix.<flavor>.<resource>.<verb>.v1`? Or shorter `helix.<flavor>.<verb>`? Recommend the full reverse-DNS form; aligns with CloudEvents community conventions.
3. **Schema registry?** Should Herald publish JSON Schemas for each event type? Recommend: yes, at `https://herald.example.com/schemas/{type}.json`, referenced via `dataschema` attribute.
4. **Tenant in source URI vs subject?** Recommend `source = https://pherald.example.com/tenants/{tenant_id}` + `subject = <resource_id>`. Documents in spec V3 §4z.

5. **Outbound batch CloudEvents?** Some consumers prefer batched events (`application/cloudevents-batch+json`). Recommend: emit single events; batched is Wave 4c+1.

§7. References

(All fetched 2026-05-22.)

- **Spec.** CloudEvents 1.0.2 — https://github.com/cloudevents/spec/blob/v1.0.2/cloudevents/spec.md
- **HTTP binding.** https://github.com/cloudevents/spec/blob/v1.0.2/cloudevents/bindings/http-protocol-binding.md
- **JSON event format.** https://github.com/cloudevents/spec/blob/v1.0.2/cloudevents/formats/json-format.md
- **Go SDK.** https://github.com/cloudevents/sdk-go — v2.16.x (latest 2026), Apache 2.0.
- **Kafka binding (Go).** https://github.com/cloudevents/sdk-go/tree/main/protocol/kafka_sarama
- **Documentation site.** https://cloudevents.io/

§8. Catalogue-check verdict

Per §11.4.74:

- **vasic-digital:** no module. Herald's `commons/types.go` already has an ad-hoc CloudEventEnvelope (replace with SDK).
- **HelixDevelopment:** no module.

Verdict: no-match → vendor `cloudevents/sdk-go` as Herald-internal submodule under `submodules/go-cloudevents-sdk/`. Alternative `go get` carve-out per §9.1.